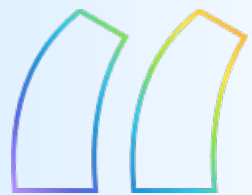




HAMI在顺丰的落地实践

顺丰科技-大数据中心-AI技术平台组
陈俊超



01 顺丰科技GPU管理技术演进



GPU算力演进历程

从独占孤岛到灵活池化:算力资源的五次变革

物理机时代

GPU整机器独占
业务直接绑定物理机
资源利用率极低

K8s集群时代

K8s纳管GPU
容器化调度
一卡一用
无切分能力
利用率瓶颈未突破

商业池化方案

实现GPU共享与切分
成本高
厂商锁定
闭源黑盒

MIG技术

硬件级别隔离
稳定性高
仅限NVIDIA高端卡

HAMI异构算力池化

任意粒度切分
多品牌统一纳管
零业务侵入
开源可控

灵活、统一、开源--当前最优解

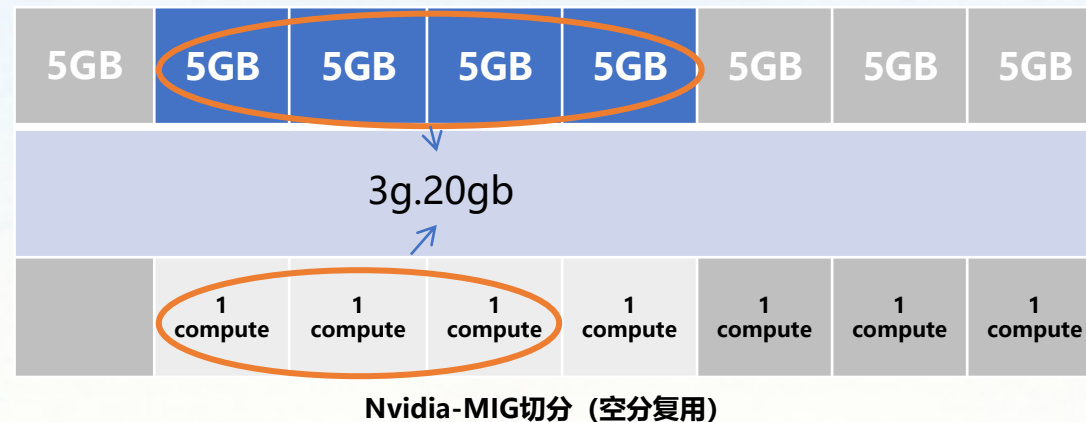
GPU池化方案选型

当前现状

存在小型推理任务占用不满整卡的情况，例如：TTS，翻译，SD，Rerank，Embedding等模型，造成资源浪费。

官方方案

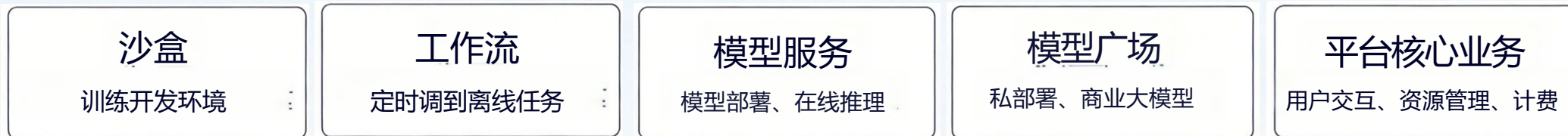
nvidia-device-plugin提供了三种共享GPU的方案：Time-Slice、MPS、MIG



方式	k8s中共享GPU	错误隔离	显存隔离	性能	支持卡型号	监控	显存超分
Time-Slice	是	否	否	context切换	nvidia任意卡	否	否
MPS	是	否	是	损耗小	V100以上	否	否
MIG	是	是	是	损耗小	A100以上	是	否

AI平台混合云架构

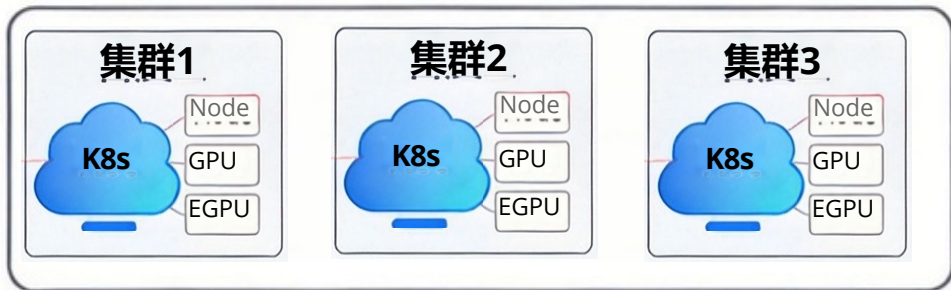
应用层



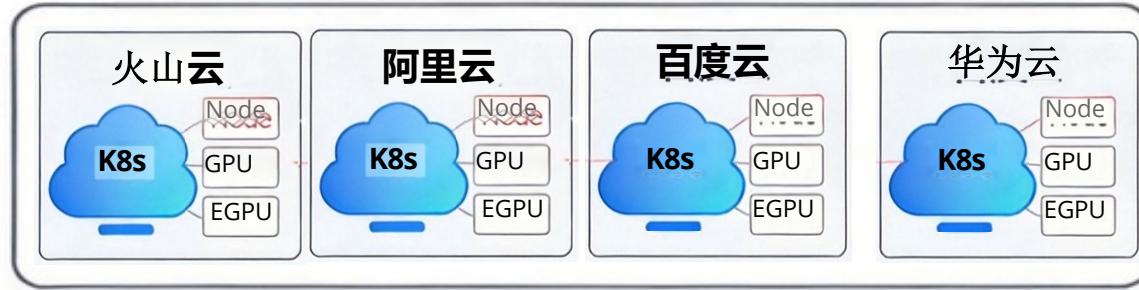
服务层



私有云



公有云



应用层抽象+底层多云K8s融合

资源管理：资源池抽象+Volcano Queue配额管理

资源池1

CPU: 100c
内存: 200g
GPU-V100: 10
GPU-A100: 10

资源池2

CPU: 100c
内存: 200g
GPU-H20: 5
GPU-L20: 10

资源池3

CPU: 100c
内存: 200g
GPU-V100: 10
VGPU-L20: 20

queue1

spec:
capability:
cpu: "100"
memory: 200Gi
nvidia.com/gpu-v100: "10"
nvidia.com/gpu-a100: "10"

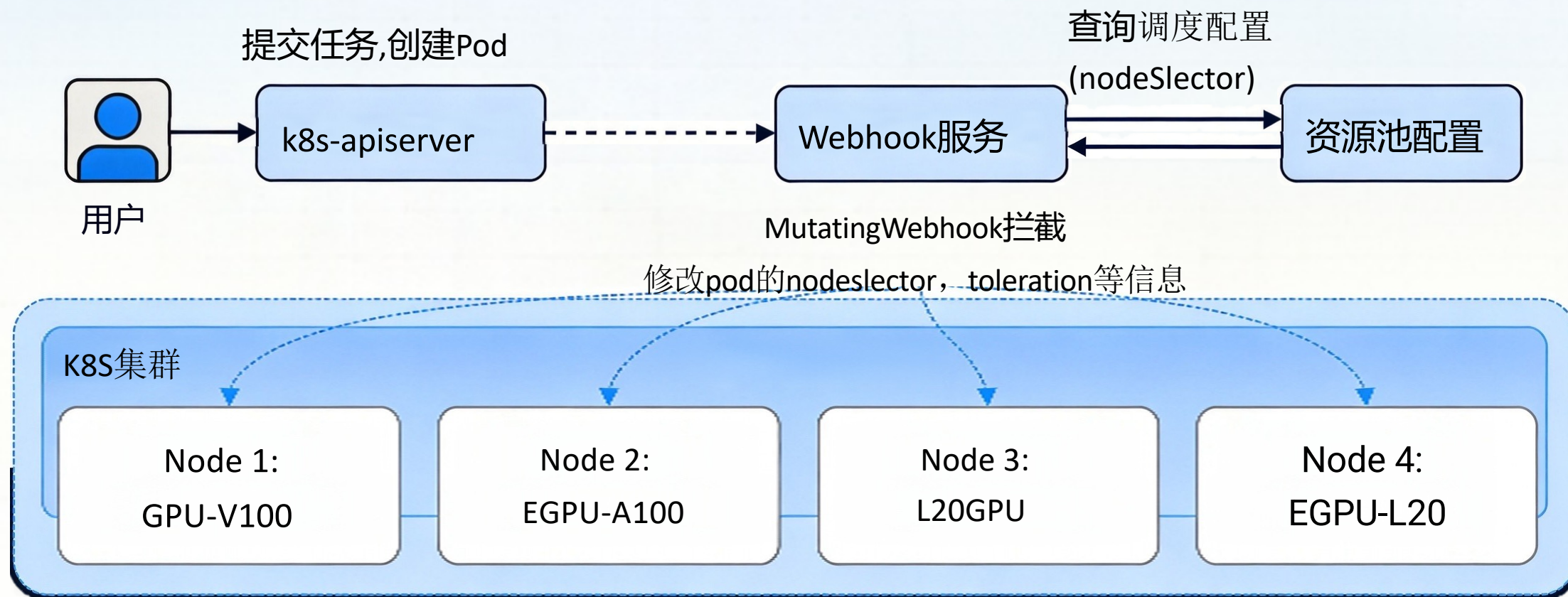
queue2

spec:
capability:
cpu: "100"
memory: 200Gi
nvidia.com/gpu-H20: "5"
nvidia.com/gpu-L20: "10"

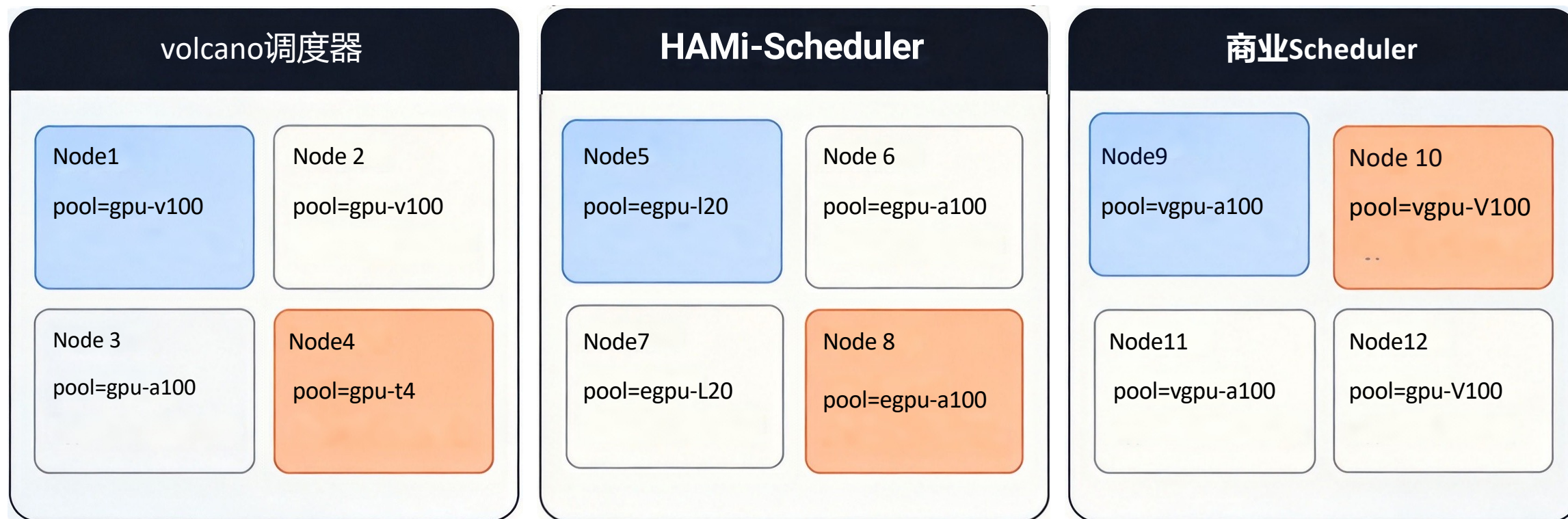
queue3

spec:
capability:
cpu: "100"
memory: 200Gi
nvidia.com/gpu-v100: "10"
volcano.sh/vgpu-number-l20: "20"

资源调度：资源池+Webhook拦截修改



资源调度阶段一：按型号打标签,多调度器管理不同节点



运维复杂: 需要按型号维护各种标签
配置复杂: 每种型号需要单独配置

调度器: 多调度器并存,难以统一管理
利用率低: CPU和内存无法跨节点共享

资源调度阶段二:DevicePlugin统一上报+Volcano Queue统一管理

1 Device Plugin统一上报

资源型号统一上报给kubelet
nvidia.com/gpu-A100、
nvidia.com/gpu-V100、
nvidia.com/gpu-L20

2 统一节点标签

消除节点标签和污点

3 Webhook智能注入

用户提交任务到资源池
Webhook根据GPU类型自动在pod上设置gpu resourcename



Volcano Queue统一管理

```
apiVersion: scheduling.volcano.sh/v1beta
kind: Queue
metadata:
  creationTimestamp: "2025-01-15T12:24:4
  generation: 88
  name: volcano-queue-6098
  resourceVersion: "2290713844"
  selfLink: /apis/scheduling.volcano.sh/
  uid: 46390a80-6875-4ca5-9756-bd2594274
spec:
  capability:
    cpu: "870"
    memory: 3170Gi
    nvidia.com/gpu-a100: "34"
    nvidia.com/gpu-a800: "14"
    nvidia.com/gpu-p40: "2"
    nvidia.com/gpu-v100: "12"
    nvidia.com/gpu-v100-16g: "8"
    nvidia.com/mig-3g.40gb: "0"
    volcano.sh/vgpu-number-a800: "60"
  guarantee: {}
  reclaimable: false
  weight: 1
status:
```

优点

资源管理统一:Volcano Queue管理所有类型GPU资源
运维成本低:无需按型号维护标签

消除标签和污点:资源统一调用

统一抽象层+单一调度器=运维简化,资源融合

Nvidia Device Plugin改动

增加gpu resource name映射

```
apiVersion: v1
kind: ConfigMap
data:
  filedata: |
    version: v1
  flags:
  resources:
    gpus:
      - pattern: 'NVIDIA A100'
        name: nvidia.com/gpu-a100
      - pattern: 'NVIDIA A800-SXM4-80GB'
        name: nvidia.com/gpu-a800
      - pattern: 'NVIDIA H20'
        name: nvidia.com/gpu-h20-96g
```

```
Allocatable:
  cpu:                216
  ephemeral-storage:  3799713271847
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:              506331744Ki
  nvidia.com/gpu:      0
  nvidia.com/gpu-h20-96g: 8
  pods:               200
```

Volcano Vgpu Device Plugin 改动

修改社区代码，增加gpu resource name 映射功能

```
apiVersion: v1
kind: ConfigMap
data:
  nvidia: |
    resourceCountName: volcano.sh/vgpu-number
    resourceMemoryName: volcano.sh/vgpu-memory
    resourceMemoryPercentageName: volcano.sh/vgpu-memory-percentage
    resourceCoreName: volcano.sh/vgpu-cores
    nvidiaResource:
      - pattern: 'NVIDIA H20-3e'
        name: volcano.sh/vgpu-number-h20-141g
      - pattern: 'NVIDIA H20'
        name: volcano.sh/vgpu-number-h20-96g
      - pattern: 'NVIDIA L20'
        name: volcano.sh/vgpu-number-l20
```

```
Allocatable:
  cpu:                177341m
  ephemeral-storage:  1948005369908
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:              985942279308444m
  nvidia.com/gpu-l20:  0
  pods:               211
  vke.volcengine.com/eni-ip: 210
  volcano.sh/vgpu-cores: 800
  volcano.sh/vgpu-memory: 368544
  volcano.sh/vgpu-number-l20: 80
```

调度策略选择

Binpack策略

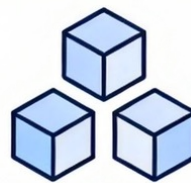


资源集中

- 1、优先集中到少数节点
- 2、提升资源利用率

当前选择

Spread策略



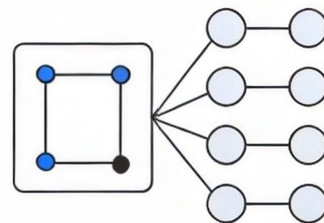
高可用负载

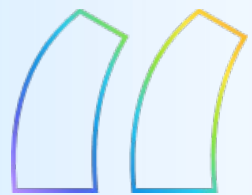
- 1、优先分散到不同节点
- 2、高可用保障
- 3、负载均衡

高可用保障-产品层方案

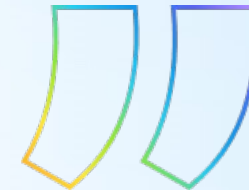
模型服务组

在网关层面,将流量同时分发到多个集群





02 GPU池化落地场景



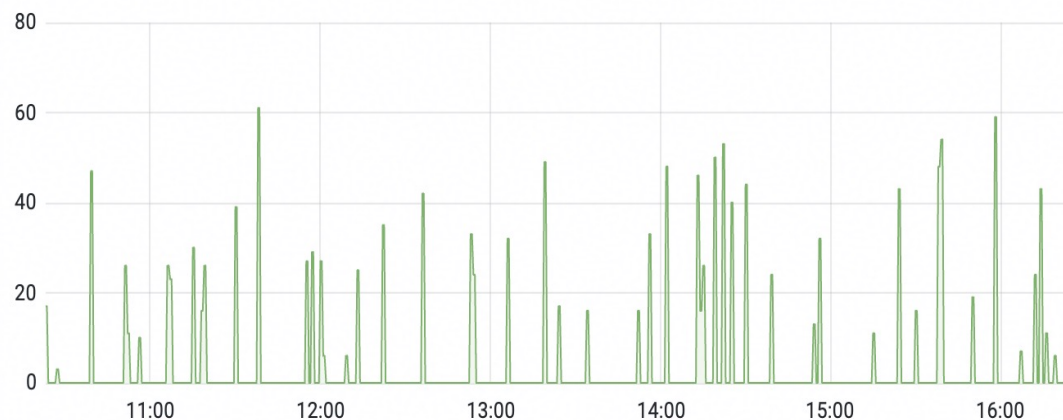
任务特征

- ◆ 显存利用率不高
- ◆ 算力利用率不高
- ◆ 存在大量资源浪费的情况

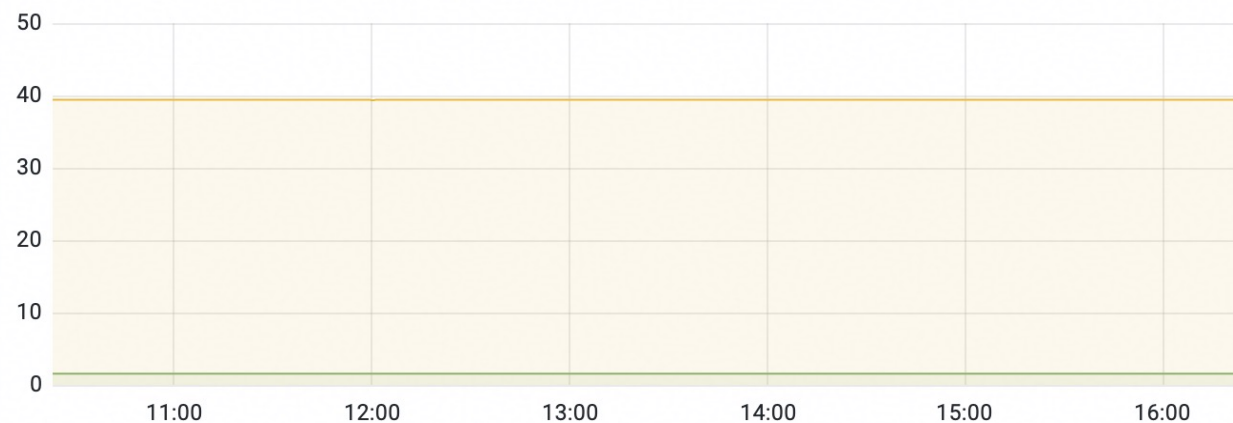
提升方案

- ✓ 使用切分的方式
- ✓ 将空闲的资源节省出来给其他任务使用
- ✓ 减少算力浪费的情况。

GPU算力



GPU显存(GB)



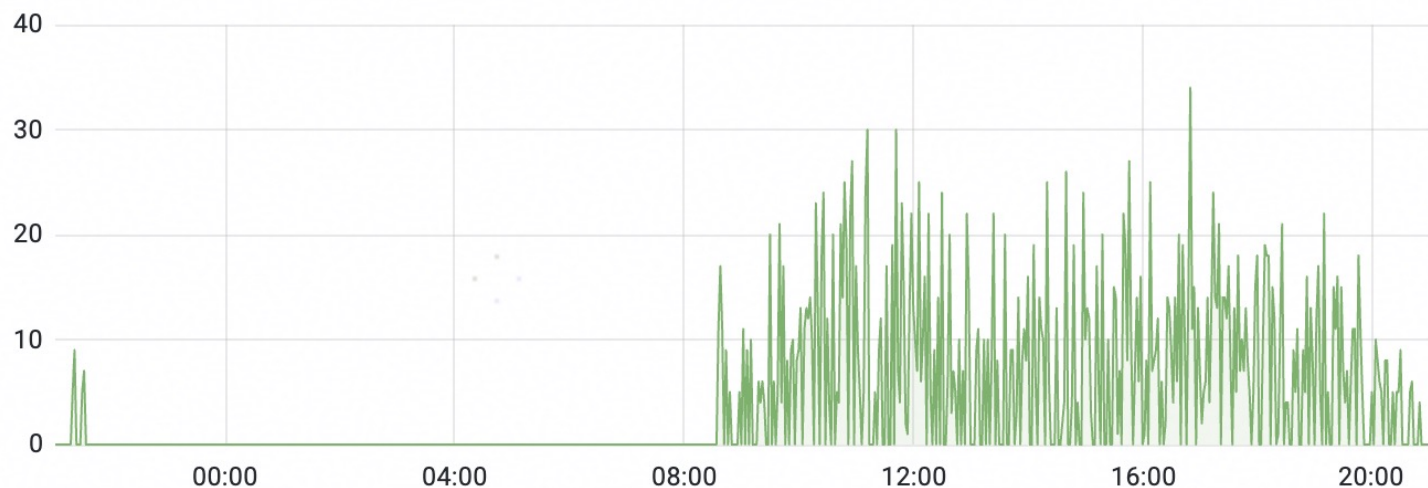
任务特征

- ◆ 工作时间活跃
- ◆ 傍晚无调用
- ◆ 算力利用率不高

提升方案

- ✓ 设置低优先级训练任务，设置推理任务为高优先级
- ✓ 将两个任务叠加到同一张卡
- ✓ 当有推理任务调用时，算力优先保障推理任务

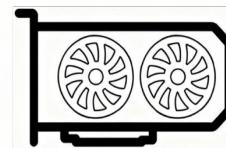
EGPU平均GPU利用率



高优先级推理

低优先级训练

EffectiveGPU混部



任务特征

- ◆ 任务一、任务二有错峰使用的情况
- ◆ 或调用量不大，没有大量并发请求推理的情况
- ◆ 且显存需求都很大
- ◆ 错峰调用场景适配

提升方案

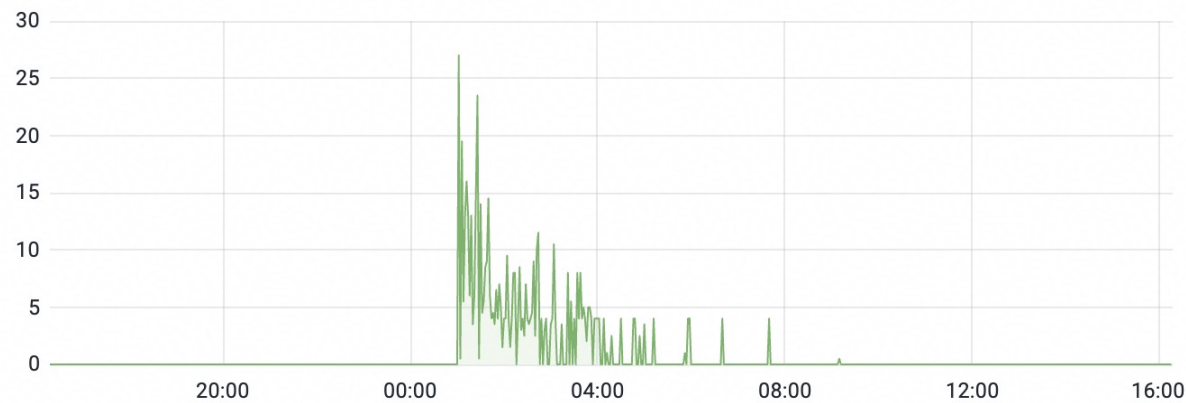
- ✓ 使用显存超分的方式
- ✓ 将任务一二部署到同一张卡上
- ✓ 提升算力利用率

EGPU平均GPU利用率



任务一

EGPU平均GPU利用率



任务二

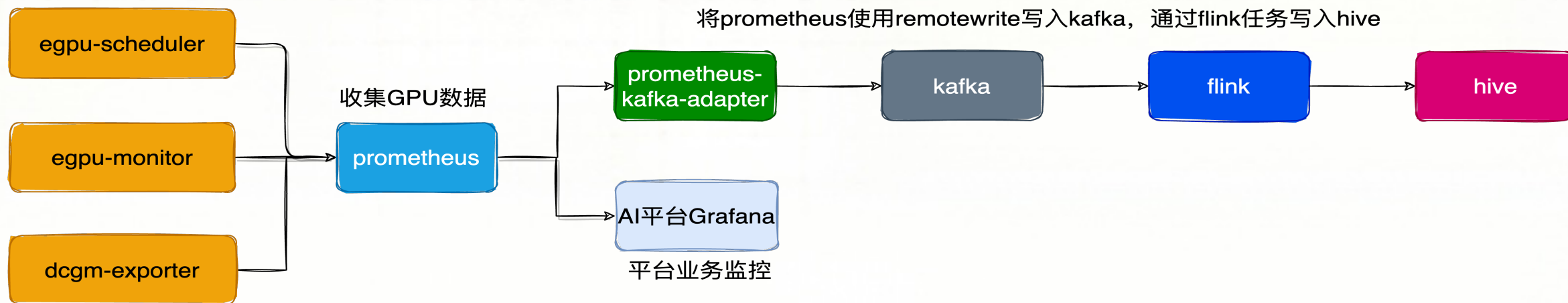
Effective GPU 监控

egpu-scheduler

- GPU卡级别显存分配量
- GPU卡级别算力分配量
- Pod服务级别显存分配量
- Pod服务级别算力分配量
- GPU卡上共享的Pod服务数量

device-plugin

- 节点GPU卡级别实时显存利用率
- 节点GPU卡级别算力实时利用率
- Pod服务级别显存实时利用率
- Pod服务级别算力实时利用率



EffectiveGPU白皮书：大模型时代，如何更好地提升算力效率？

<https://mp.weixin.qq.com/s/zZKJfJJOzBhpcq1FMttqsA>